PetCare Clinic viva guide

Enterprise Application Development 1 | DSE 25.3

Use this guide alongside the source project. Practise the workflow, point to the relevant code and explain each decision in your own words. The application still needs MySQL, JasperReports and Windows execution checks on your computer.

Understand the project well enough to change a validation rule or SQL query during the viva. Memorizing these answers alone will not prepare you to demonstrate the code.

Two minute introduction

"My application is a Java Swing veterinary clinic system. The main workflow is booking an appointment, recording treatment and collecting payment. Customers can have several pets. A pet receives a service from a vet at a fixed 30-minute slot. MySQL stores the records, and a Jasper report combines appointments with customer, pet, service, vet, treatment and payment data. I separated UI code, business rules and database access so each responsibility is easy to locate. Treatment and payment updates use database transactions to avoid partial records."

Use your own words and be ready to point to the actual classes.

Questions and answers

Why did you choose this scenario

A clinic needs to connect the owner, pet, booking, treatment and payment. This creates a meaningful transaction rather than a collection of unrelated registration screens. It directly fits the specified pet care and veterinary services sector.

Which design patterns are implemented

The DAO pattern appears in ClinicDao. It keeps SQL, PreparedStatement binding and ResultSet processing outside the screens. Database.open is a static connection factory: callers ask it to create a configured JDBC Connection. The UI, service and DAO separation is a layered architecture. ClinicFrame contains both view and event handling; do not claim this is a strict implementation of MVC with a separate controller class. Do not claim a Singleton is used: each operation obtains a fresh connection.

How does appointment booking work

ClinicFrame validates the selected time and calls ClinicService.book. The service validates it again, starts a transaction, locks the selected vet's row and checks for an occupied slot. It inserts the appointment using the active service's current price and the pet's current customer. MySQL's unique constraints provide a final protection for both the vet and pet. The transaction commits only after insertion succeeds.

Why lock a vet row

An empty appointment slot has no appointment row to lock. Locking the existing vet row gives concurrent bookings for the same vet a common lock. A second transaction waits and then sees the first booking. The database unique index protects the pet even when two transactions book it under different vets.

What happens when an appointment is cancelled

Only BOOKED appointments can be cancelled. The generated occupied_slot column becomes NULL. MySQL unique indexes allow multiple NULL entries, so the slot can be booked again without deleting the cancelled record. Completed appointments retain their occupied slot. Cancelled appointments contribute no outstanding balance to the UI or report.

Why capture the fee and customer at booking

A service price may change later. The appointment fee should remain the price agreed at booking. A pet may move to a different owner; the customer reference on the appointment preserves the customer responsible for that visit. Customer and service display names are still read from their current records; this application does not store complete historical name snapshots.

How is treatment saved safely

The service locks the appointment and verifies that it is BOOKED. It validates the initial payment against the stored fee, inserts one treatment, optionally inserts a payment and marks the appointment COMPLETED. All steps use the same connection with auto-commit disabled. If an SQL or validation error occurs, rollback reverses the changes. The treatment table also has a unique appointment_id to prevent two treatments for one appointment.

How do later payments prevent overpayment

The service locks the appointment, reads its fee and totals existing payments. It accepts only a positive payment no greater than the remaining balance. Concurrent payments use the same appointment lock, so they cannot independently spend the same balance.

Why use BigDecimal

Binary floating-point values can introduce rounding differences. BigDecimal represents decimal currency values exactly. Validation permits at most two decimal places, and MySQL stores amounts in DECIMAL(10,2). Payment values are never calculated using double.

What is the custom exception

ValidationException represents an expected user input or business rule problem. It provides a readable message, such as an occupied slot or excessive payment. SQLException represents database failures. ClinicFrame handles them separately, shows a useful message and logs diagnostic database details.

What prevents SQL injection

User values go into PreparedStatement parameters. They are bound as data rather than concatenated into SQL syntax. The delete table name comes from a fixed switch in the application, not from user input. SQL identifiers cannot be replaced by ordinary JDBC placeholders.

How does Swing remain responsive

Main starts the UI using SwingUtilities.invokeLater. The reusable task method uses SwingWorker: doInBackground runs database and report work away from the event dispatch thread; done returns results to Swing. A busy flag prevents overlapping operations from the same window. Swing components are updated from the event dispatch thread.

What tables are used in the Jasper report

Appointments, pets, customers, services, vets, treatments and payments. Payments are grouped by appointment in a subquery before the join. This avoids duplicating an appointment row when it has several payments, which would otherwise inflate fee totals or visit counts. Treatments use a LEFT JOIN so untreated visits still appear.

Explain report compilation filling and exporting

The JRXML file describes fields, parameters, SQL and page layout. JasperCompileManager compiles it into a JasperReport. JasperFillManager runs its parameterized SQL using the database connection and produces a JasperPrint. JasperExportManager writes the filled document as a PDF.

Why use a half-open date range

The report accepts inclusive start and end dates in the UI. The service sends start-date midnight and midnight after the end date. The query uses slot >= start and slot < next-day midnight. This includes the entire final day without depending on fractional-second precision.

How does dashboard revenue differ from report payments

Dashboard receipts are filtered by payment date. The report is filtered by appointment date and shows every payment received for those visits, even if a payment was made later. It is a visit-based revenue and balance report, not a payment-date cashbook.

Explain the database relationships

A customer has many pets and may be responsible for many appointments. A pet has many appointments. Each appointment references one service and one vet. An appointment has zero or one treatment and may have several payments. Foreign keys prevent orphan records. Deleting a referenced customer, pet or service is restricted to protect history.

Is every table in third normal form

Most descriptive data is separated into dedicated entities. Appointment fee and customer_id intentionally preserve booking facts, rather than always deriving them from current service or pet records. Explain this as historical booking data and avoid an unsupported claim that every field is always determined only by its present-day related entity.

Where are credentials stored

In environment variables. Database.open reads them when opening a connection. The source does not contain a database password. The dedicated application account is granted data operations only, not schema administration.

How do login registration and logout work

AuthPanel provides sign-in and registration forms. AuthService validates usernames and passwords, saves account records using prepared statements and verifies credentials. PasswordHasher uses PBKDF2-HMAC-SHA256 with 600000 iterations, a random 16-byte salt and a 256-bit hash. It compares derived hashes with MessageDigest.isEqual and clears temporary password character arrays. Registration opens the clinic workspace. Logout disposes that workspace and opens a fresh sign-in window. It is blocked while a database task is running. All registered staff have the same access; role authorization is not implemented.

Why is a salt needed

Different random salts cause the same password to produce different stored hashes. The salt is stored with the hash and iteration count so login can repeat the derivation. Passwords are hashed rather than reversibly encrypted or stored as plaintext. PBKDF2 is available in the JDK, so this coursework avoids an additional password library.

How did you improve the user interface

Theme centralizes typography, colours, button styles and table formatting. The sidebar gives consistent navigation. DashboardPanel shows four summary cards and a daily appointment list. The login panel provides clear labels, password visibility, validation feedback and a separate registration mode. Database and password-hashing work run on SwingWorker so the interface can remain responsive.

Is the authentication production ready

No. This local coursework has open staff registration, equal privileges, no password recovery or MFA and a window-local failed-login cooldown. A production system needs administrator-controlled registration, server-enforced permissions, persistent rate limiting and auditing. A user with database credentials can bypass the desktop login; protecting the database is a separate requirement.

Why are email and SMS absent

API integration is optional in the brief. This version focuses on the required clinic workflow, reporting and the requested account screens. Notifications can be added after verifying the core behavior.

How do JAR and EXE differ

The shaded JAR contains Java classes and library dependencies; a compatible Java runtime runs it. The Windows application image contains an EXE launcher, the JAR and a bundled runtime. Its EXE must be distributed with the complete image folder. MySQL is not embedded in either output.

Live demonstration order

1. Register a staff account, log out and sign in again. Then show the database tables, relationships and source tree.
2. Refresh Dashboard and explain its values.
3. Add a customer; demonstrate blank-name and invalid-phone validation.
4. Add their pet; reject a future birth date.
5. Add a service and book a future 30-minute appointment.
6. Try the same vet and slot, then the same pet and slot with another vet.
7. Complete treatment with a partial payment; open the treatment details.
8. Try a payment greater than the balance; then pay the exact balance.
9. Export the report covering the appointment date and check its totals.
10. Try to delete a customer with a linked pet and explain the foreign key restriction.
11. Show the JAR, launch the Windows EXE and open your GitHub commits.

Practice code changes

- Change the maximum name length consistently in validation and schema, and explain why both matter.

- Add a species filter to the pets query using a PreparedStatement parameter.

- Add a new service using the UI and show it in the booking selection.

- Explain why removing rollback would corrupt transaction consistency.

- Identify the bug caused by joining raw payment rows directly to appointments.

What you must know before submission

Be able to explain try-with-resources, prepared statements, foreign keys, indexes, BigDecimal, auto-commit, commit, rollback, row locking, the Swing event dispatch thread, SwingWorker, JRXML and the generated occupied_slot expression. Run the test checklist yourself and record actual results. Do not claim any test passed until you have observed it.

Verification before submission

The Java UI and core service source compiled without warnings using a temporary report type-check substitute. Twenty-five real validation checks passed. The full Maven build, live MySQL transactions, actual Jasper export, Windows GUI and EXE remain to be verified. Refer to docs/VERIFICATION.md and docs/TEST_CHECKLIST.md in the project.